Phase 2

Backend Development and API Integration

User and Product Management API

Phase 2 Backend Team
IS Elec 2: Web Systems and Technologies

Project scope

Two resources

Users store account details. Products store item details, price, and stock.

Four database operations

Create a record, retrieve it, change its values, and delete it.

Visible backend output

HTTP status codes and JSON responses show what the server did.

How one request reaches the database

1. Client and route

Postman or the demo script sends an HTTP request. Express selects the route.

2. Validation and controller

Middleware checks input. The controller performs the requested operation.

3. Model, database, and response

Mongoose works with MongoDB. Express returns a status code and JSON.

Backend folder structure

`backend/`	
`  server.js`	Startup and Express setup
`  app.js`	Database connection and schemas
`  config/`	
`  models/`	Endpoints and database operations
`  routes/`	
`  controllers/`	Input checks and error responses
`  middleware/`	
`  scripts/`	Demonstration and automated tests
`  tests/`	

Server startup

```
require('dotenv').config({
  path: require('node:path').join(__dirname, '.env')
});
const connectDB = require('./config/db');
const app = require('./app');

async function start() {
  await connectDB();
  const port = Number(process.env.PORT || 5000);
  const server = app.listen(port, '127.0.0.1', () => {
    console.log(`Server running on port ${port}`);
  });
```

Startup excerpt. Full error and shutdown handling remains in server.js.

MongoDB connects before Express starts listening.

MongoDB connection

```
await mongoose.connect(process.env.MONGO_URI, {
  serverSelectionTimeoutMS: 5000
});
await Promise.all([
  require('../models/User').init(),
  require('../models/Product').init()
]);
console.log('MongoDB Connected');
```

.env.example documents the settings. The local .env stays out of Git.

User and Product models

User

```
password: {
  type: String,
  required: true,
  select: false
},
role: {
  type: String,
  enum: ['user', 'admin'],
  default: 'user'
}
```

Product

```
price: {
  type: Number,
  required: true,
  min: 0
}
```

Stock must also be a whole number of zero or more.

Schema excerpts. Full definitions are in backend/models/.

Routes select the operation

```
router.route('/')
  .post(validateBody(fields, ['name', 'email', 'password']), controller.create)
  .get(controller.list);
router.route('/:id')
  .all(validateId)
  .get(controller.get)
  .put(validateBody(fields), controller.update)
  .delete(controller.remove);
```

Example: PUT /api/users/:id calls controller.update after validation.

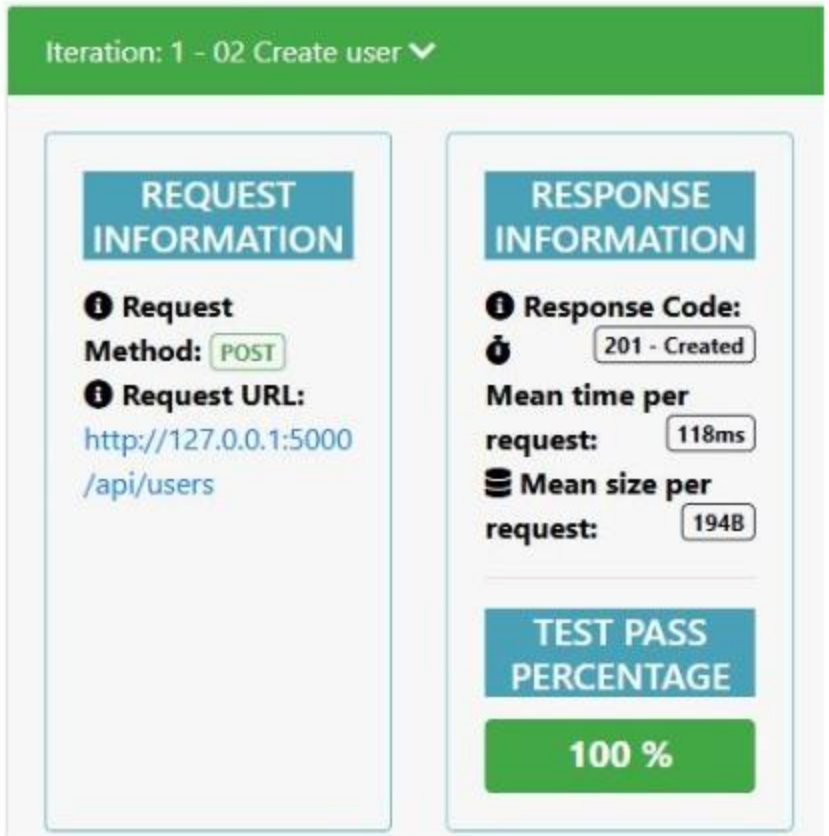
The controller saves the change

```
update: async (req, res) => {
  const resource = await find(req.params.id);
  Object.assign(resource, await prepare(req.body));
  await resource.save();
  res.json(resource);
},
```

A saved update has two checks

The PUT response shows the new value. A later GET confirms the database kept it.

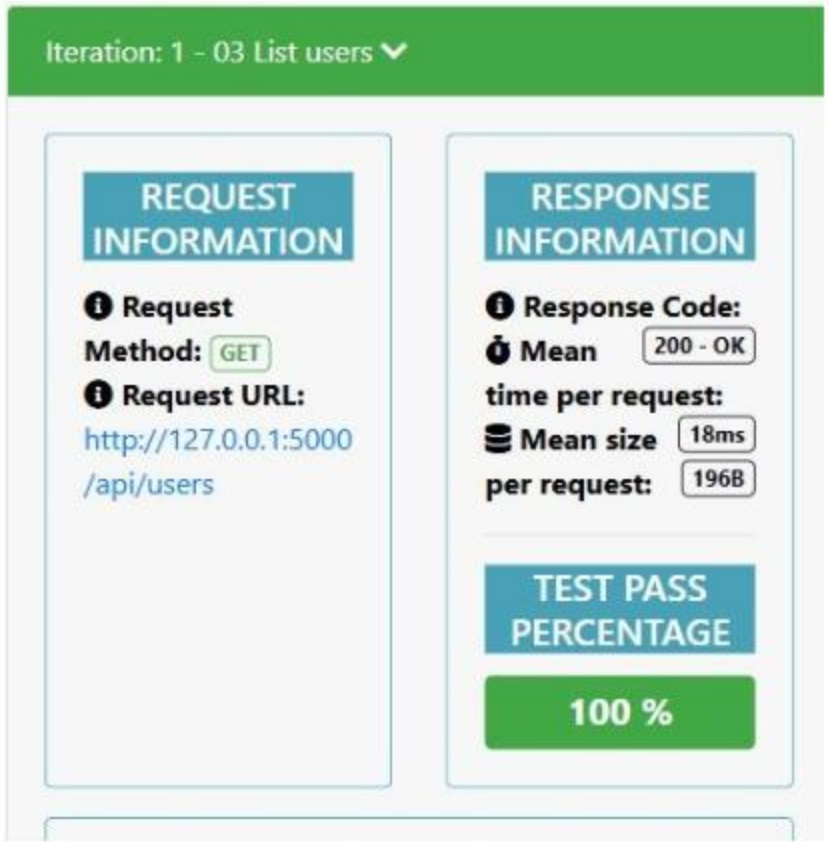
Create a user: 201 Created





The server returns a saved ID and account details. The password stays hidden.

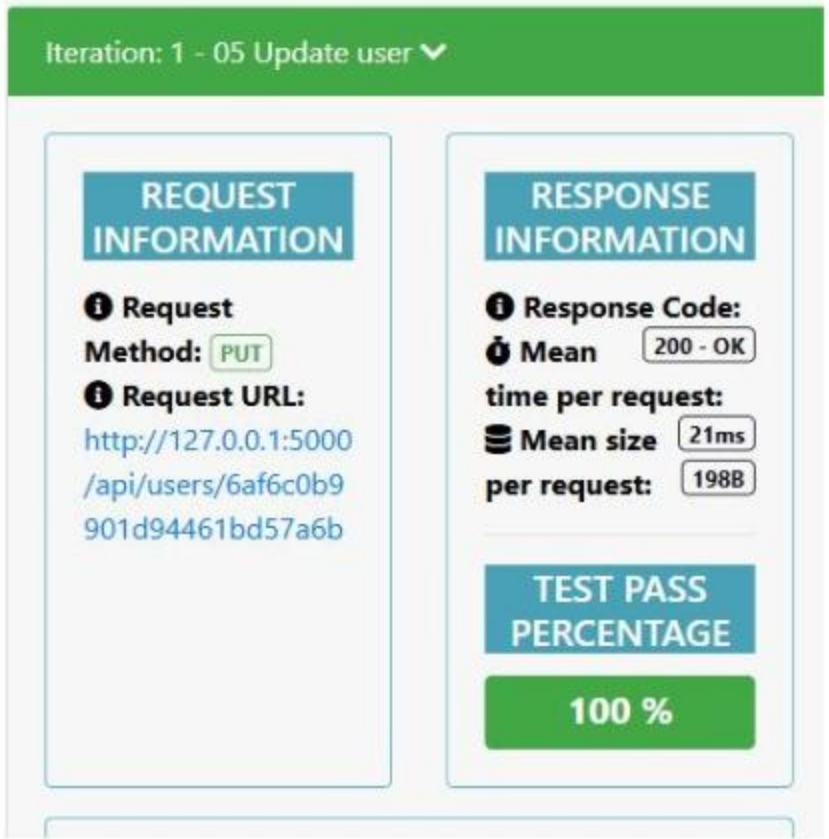
Read users: 200 OK

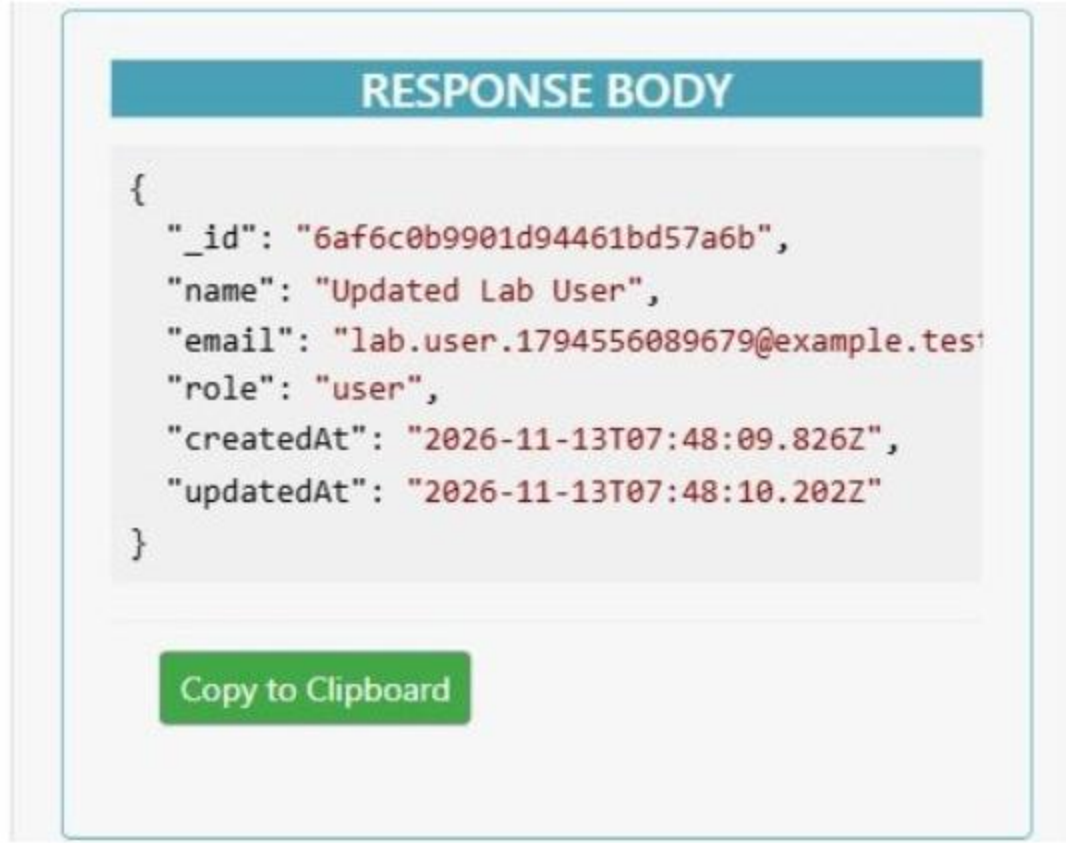




GET /api/users returns an array that includes the saved user.

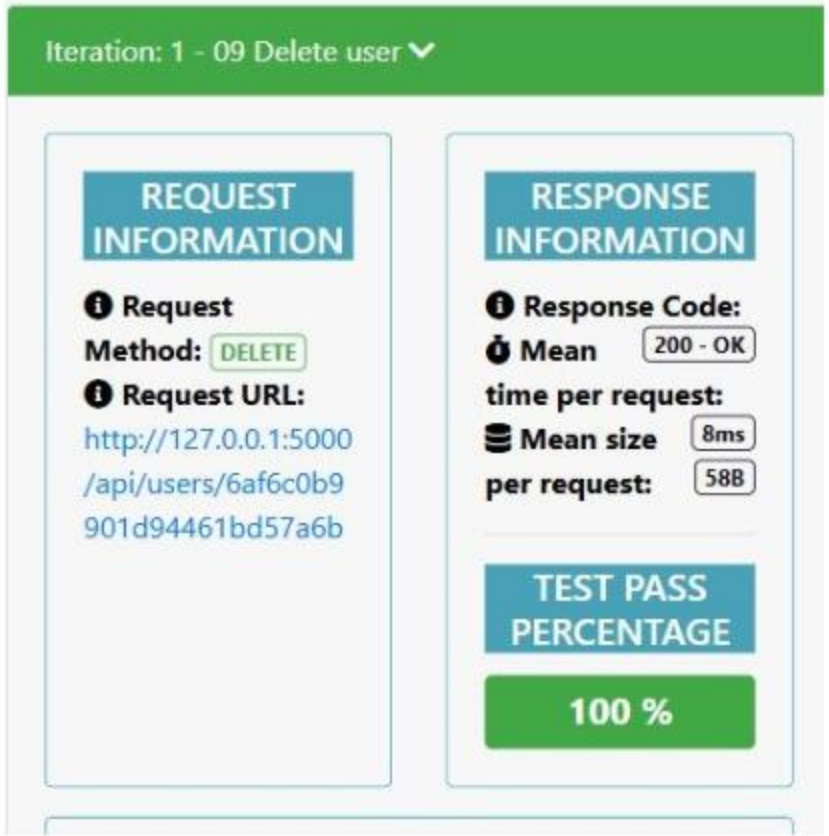
Update a user: 200 OK

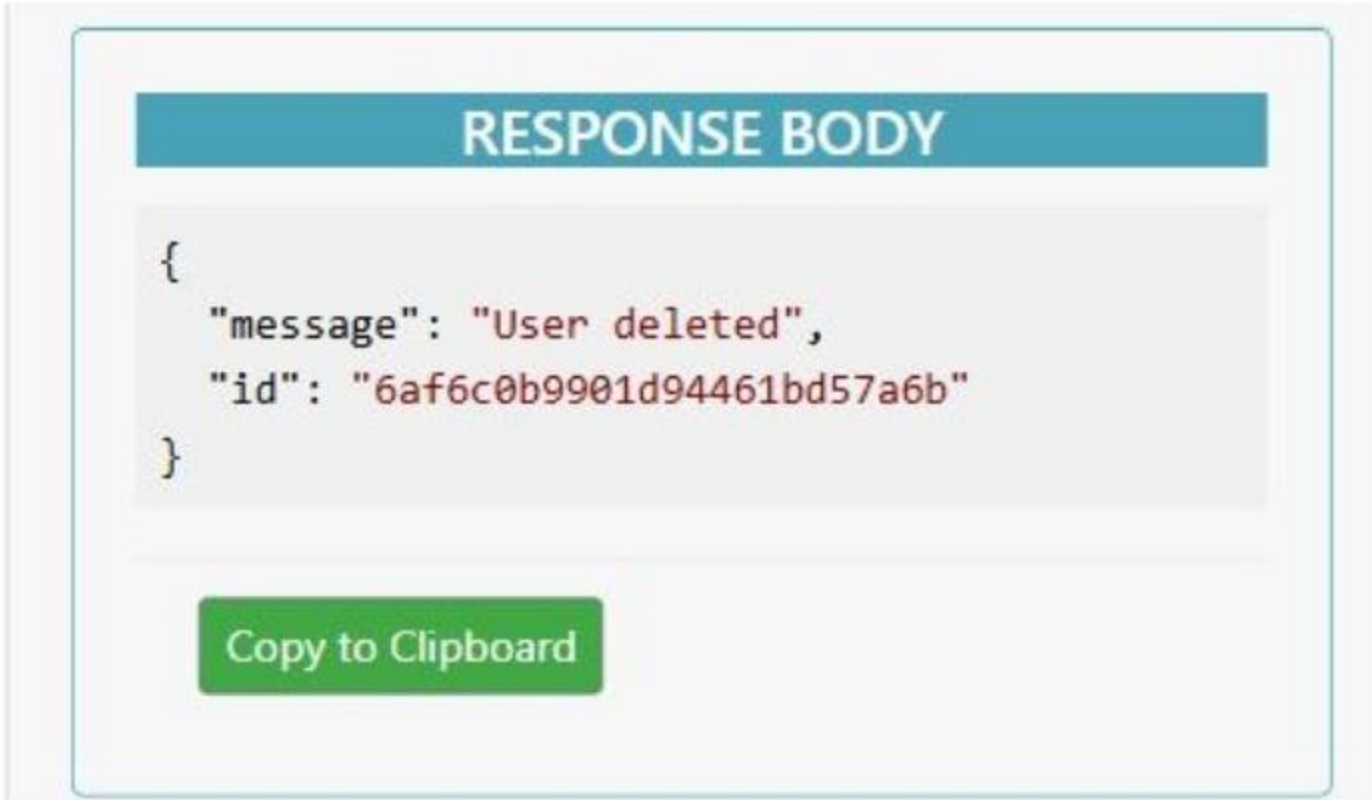




The returned name changes to Updated Lab User.

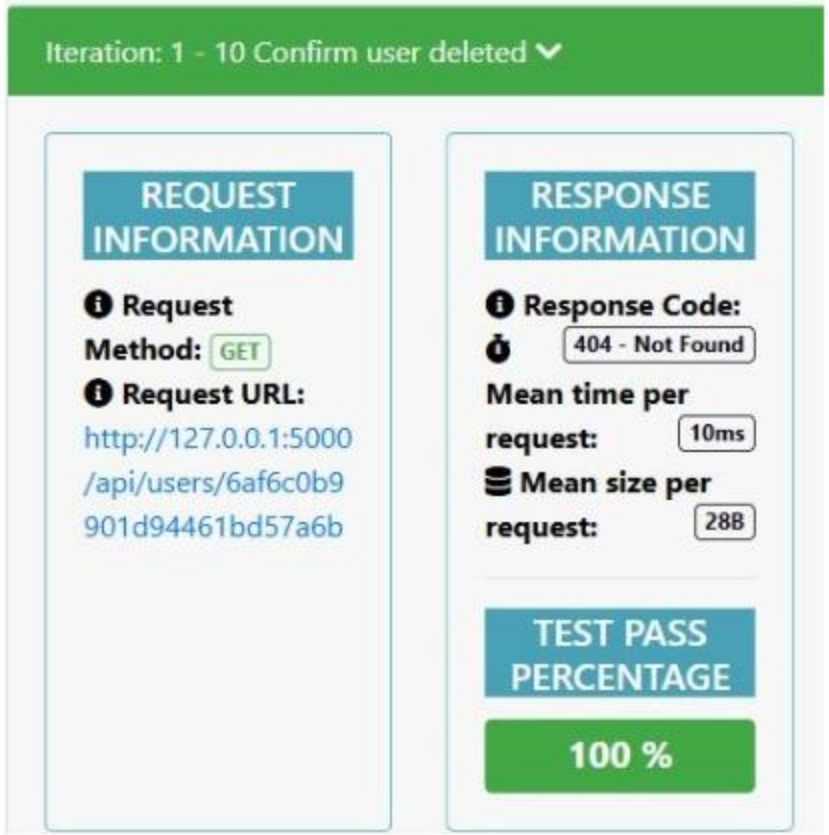
Delete a user: 200 OK





The deletion response identifies the exact user that the server removed.

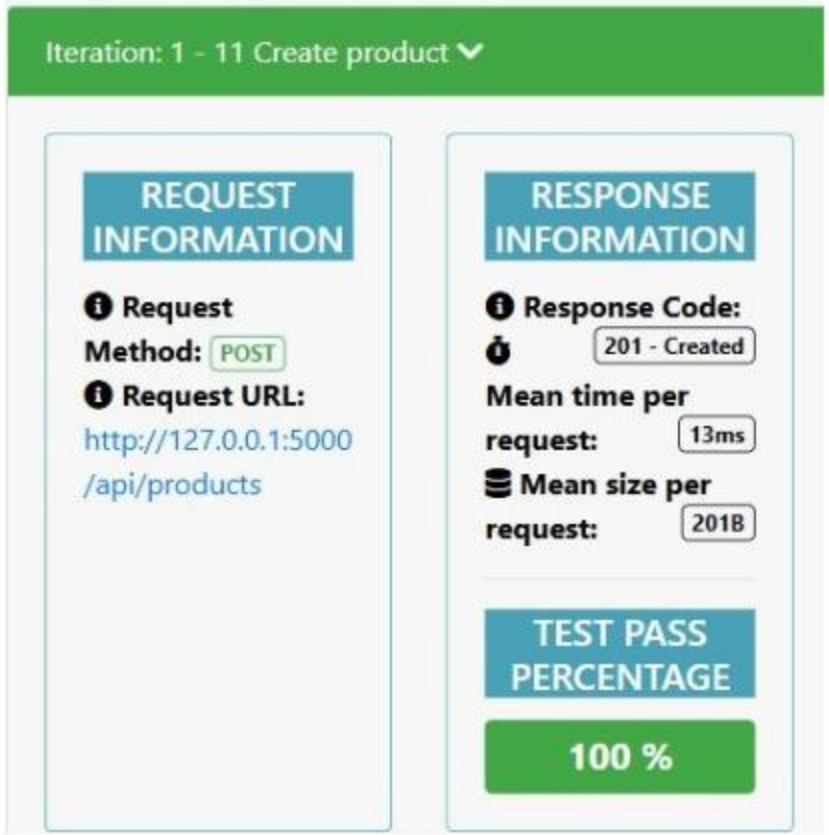
A second read confirms deletion





GET for the deleted ID returns 404 and User not found.

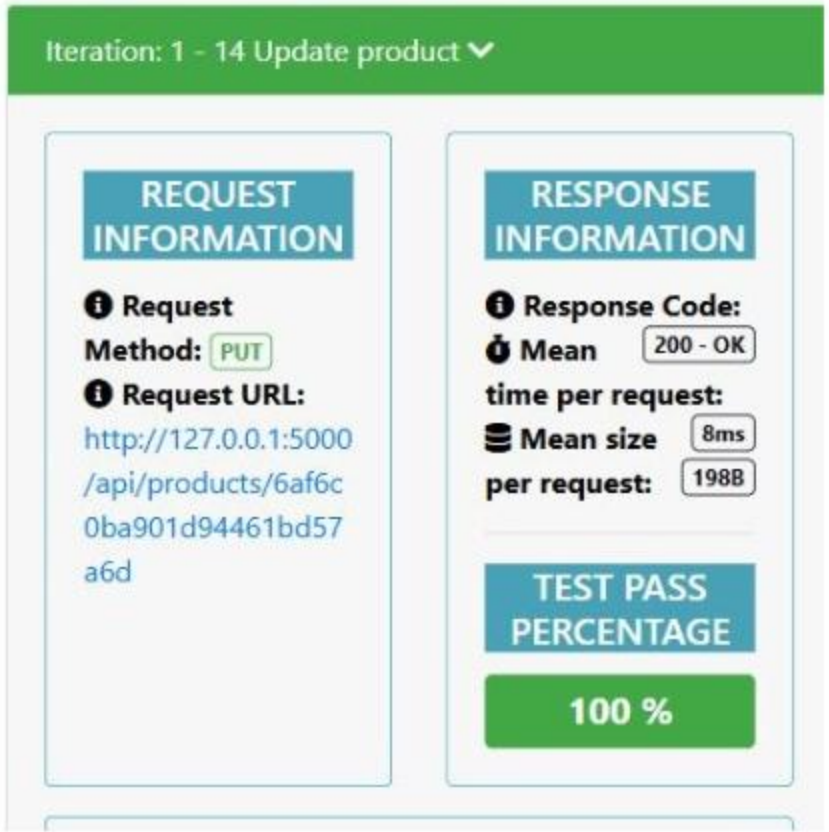
Products use the same CRUD pattern





POST creates Lab Notebook with a price of 45.5 and stock of 10.

Product changes persist





The updated product has a price of 50 and stock of 8.

Recorded test results

37

integration tests passed

20 requests
52 assertions
0 failures

SUMMARY ITEM	TOTAL	FAILED
Requests	20	0
Prerequest Scripts	1	0
Test Scripts	20	0
Assertions	52	0
Skipped Tests	0	-

Saved test evidence, 13 November 2026

Validation and error handling

400: invalid input

The API rejects malformed IDs, missing fields, and invalid values.

409: duplicate email

The unique email index prevents two users from sharing one email.

404: record not found

A deleted or nonexistent record returns a clear JSON message.

GitHub walkthrough

1. README and backend folder

Explain the project, setup commands, and separation of responsibilities.

2. Report and evidence

Open the lab report, screenshots, Postman collection, and raw results.

3. Commit history

Show the implementation, testing, and documentation changes.

Open the GitHub repository

Commits show the project history

3b5c316: backend implementation

Express setup, Mongoose models, and CRUD routes.

2e5ade4: API tests

Integration tests and the reusable Postman collection.

4bedd9e: completed backend report

Code explanations, screenshots, and recorded JSON output.

Local demonstration setup

Preparation

```
cd backend
npm ci
Copy-Item .env.example .env
```

Three terminals, all inside backend

```
1   npm run db:local
2   npm start
3   npm run demo
```

Live backend demonstration

Health check

GET /api/health should return 200 with database: connected.

User sequence

Create 201, read 200, update 200, delete 200, then read 404.

Product sequence

Repeat the same operations and explain the changes to price and stock.

Postman demonstration

Import the provided files

Phase2.postman_collection.json and Local.postman_environment.json.

Run requests in numbered order

The collection carries the created IDs into later requests.

Explain the result

Show the HTTP status, returned JSON, and passing assertions.

Presentation responsibilities

Bea Roshan Pabilona

Project management, documentation, and results

Christian Balauro

Backend code and live API demonstration

Simranjit Singh

Database connection, schemas, and persistence

Karl James Matias

GitHub repository and commit history as kjamesss

What the activity demonstrates

Working backend and database

Express routes use Mongoose models to store and retrieve MongoDB documents.

Verified CRUD behavior

Actual responses show successful operations and expected error cases.

Reproducible submission

The repository includes setup instructions, tests, and the complete report.

Questions your professor may ask

How do you prove persistence?

Read the same ID again after an update or deletion.

Is the HTML report the application frontend?

It displays results from Newman, the Postman collection runner.

Does the role field provide access control?

It stores a value. Login and permission checks are outside this phase.

Demo recovery

Connection refused or startup failed

Check the database terminal, API terminal, and local .env settings.

409 on create

Use a fresh email address. The unique index is working.

Live setup unavailable

Open backend-demo.txt and the saved report. Identify them as recorded results.